SET 1 - 3

1. Chocolate Factory

Algorithm

1. Traverse array.
2. Move all non-zero elements to the front.
3. Fill remaining positions with 0.
4. Print array.

Java Code

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int index = 0;
        for(int i = 0; i < n; i++) {
            if(arr[i] != 0) {
                arr[index++] = arr[i];
            }
        }
        while(index < n) {
            arr[index++] = 0;
        }
        for(int x : arr) {
```

```
            System.out.print(x + " ");
        }
    }
}
```

---

2. Toggling Bits

Algorithm

1. Convert number to binary.
2. Toggle every bit using XOR with mask.
3. Mask = (1 << number_of_bits) - 1.
4. Print result.

Java Code

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int bits = Integer.toBinaryString(n).length();
        int mask = (1 << bits) - 1;
        int result = n ^ mask;
        System.out.println(result);
    }
}
```

---

3. Count Sundays

Algorithm

1. Store day numbers.
2. Find days needed to reach first Sunday.
3. Count Sundays within given N days.

Java Code

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String day = sc.next().toLowerCase();
        int n = sc.nextInt();
        String[] days = {"sun","mon","tue","wed","thu","fri","sat"};
        int start = 0;
        for(int i = 0; i < 7; i++) {
            if(days[i].equals(day)) {
                start = i;
                break;
            }
        }
        int firstSunday = (7 - start) % 7;
        int count = 0;
        if(firstSunday < n) {
            count = 1 + (n - firstSunday - 1) / 7;
        }
        System.out.println(count);
    }
}
```

---

4. Risk Severity

Algorithm

1. Use Dutch National Flag Algorithm.
2. Maintain low, mid, high pointers.
3. Sort in one traversal.

Java Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int low = 0, mid = 0, high = n - 1;

        while(mid <= high) {
            if(arr[mid] == 0) {
                int temp = arr[low];
                arr[low] = arr[mid];
                arr[mid] = temp;
                low++;
                mid++;
            }
            else if(arr[mid] == 1) {
                mid++;
```

```
            }
            else {
                int temp = arr[mid];
                arr[mid] = arr[high];
                arr[high] = temp;
                high--;
            }
        }

        for(int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

---

5. Prior Elements

Algorithm

1. First element is always counted.
2. Maintain maximum seen so far.
3. If current element > maximum, increase count.

Java Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int count = 1;
```

```
        int max = sc.nextInt();

        for(int i = 1; i < n; i++) {
            int num = sc.nextInt();

            if(num > max) {
                count++;
                max = num;
            }
        }

        System.out.println(count);
    }
}
```

---

6. Pricing Format

Algorithm

1. Extract digits using modulo.
2. Multiply all digits.
3. Print product.

Java Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        long n = sc.nextLong();
```

```
        long product = 1;

        while(n > 0) {
            product *= (n % 10);
            n /= 10;
        }

        System.out.println(product);
    }
}
```

---

7. Collection of Curtains

Algorithm

1. Divide string into groups of size L.
2. Count 'a' in each group.
3. Store maximum count.

Java Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();
        int l = sc.nextInt();

        int maxCount = 0;

        for(int i = 0; i < str.length(); i += l) {
```

```
            int count = 0;

            for(int j = i; j < Math.min(i + l, str.length()); j++) {
                if(str.charAt(j) == 'a') {
                    count++;
                }
            }

            maxCount = Math.max(maxCount, count);
        }

        System.out.println(maxCount);
    }
}
```

---

8. Table Conference

Algorithm

1. President and PM treated as one block.
2. Total units = N - 1.
3. Circular arrangements = (N-2)!.
4. Internal arrangements of pair = 2!.
5. Answer = 2 × (N-2)!.

Java Code

```
import java.util.*;
import java.math.BigInteger;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();

        BigInteger fact = BigInteger.ONE;

        for(int i = 2; i <= n - 2; i++) {
            fact = fact.multiply(BigInteger.valueOf(i));
        }

        System.out.println(fact.multiply(BigInteger.valueOf(2)));
    }
}
```

---

9. Mysterious Method

Algorithm

1. If R = 0, print 0.
2. Find digit sum of N.
3. Multiply digit sum by R.
4. Repeatedly find digit sum until single digit remains.

Java Code

```
import java.util.*;

public class Main {

    static int digitSum(int n) {
        int sum = 0;

        while(n > 0) {
            sum += n % 10;
```

```
            n /= 10;
        }

        return sum;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int r = sc.nextInt();

        if(r == 0) {
            System.out.println(0);
            return;
        }

        int sum = digitSum(n) * r;

        while(sum >= 10) {
            sum = digitSum(sum);
        }

        System.out.println(sum);
    }
}
```